



Algorithms Problem Solving

Level 2

My Floor



Algorithms Problem Solving

Level 2

My Floor

Definition

- ◆ This program calculates the floor value of a number without using the built-in `floor()` function.
- ◆ The floor of a number is the greatest integer that is less than or equal to the number.
- ◆ For positive numbers, removing the decimal part gives the floor value.
- ◆ For negative numbers with a fractional part, the result must move to the next smaller integer.

Explanation

- ◆ ``ReadNumber()`` reads a floating-point number from the user.
- ◆ ``MyFloor()`` checks whether the number is positive or negative.
- ◆ If the number is positive, ``int(number)`` removes the fractional part.
- ◆ For example:
 $33.54323 \rightarrow 33$
- ◆ If the number is negative, simply converting it to ``int`` is not enough.
- ◆ For example:
 $-33.54323 \rightarrow \text{int}(-33.54323) = -33$
- ◆ But the floor of ``-33.54323`` is ``-34``, because ``-34`` is the greatest integer less than or equal to ``-33.54323``.
- ◆ Therefore, the code uses:
`--number`
- ◆ This changes the number from approximately ``-33.54323`` to ``-34.54323``.
- ◆ Converting that value to ``int`` gives ``-34``.

Steps in main()

1. Ask the user to enter a floating-point number.
2. Check whether the number is positive.
3. If positive, convert it directly to `int`.
4. If negative, decrease the number by `1`.
5. Convert the adjusted negative number to `int`.
6. Return the result.
7. Print the result from `MyFloor()`.
8. Compare it with C++'s built-in `floor()` function.

💡 Example

◆ Input:

33.54323

◆ The number is positive, so:

$\text{int}(33.54323) = 33$

◆ Therefore:

$\text{MyFloor}(33.54323) = 33$

◆ Another example with a negative number:

-33.54323

◆ The code decreases the number:

$-33.54323 - 1 = -34.54323$

◆ Then:

$\text{int}(-34.54323) = -34$

◆ Therefore:

$\text{MyFloor}(-33.54323) = -34$

➡ Result

My Floor Result: 33

C++ Floor Result: 33

Note

- ◆ Important function:

```
float MyFloor(float number)
{
    if (number > 0)
        return int(number);
    else
        return int(--number);
}
```

- ◆ `int(number)` truncates the decimal part toward zero.

- ◆ For positive numbers, truncation toward zero gives the same result as floor.

- ◆ For negative numbers with a fractional part, truncation toward zero gives a value that is too large, so the code decreases the number first.

- ◆ If the input is exactly an integer, the result remains the same:

33.0 → 33

-33.0 → -33

⊘ Warning

- ◆ The expression `int(--number)` is a clever approach for this exercise, but it modifies the local variable `number`.
- ◆ The logic relies on C++ conversion from `float` to `int`, which truncates toward zero.
- ◆ `float` has limited precision, so very precise decimal values may have small representation differences.
- ◆ `corecrt_math_defines.h` is not needed.
- ◆ `cmath` is used for the built-in `floor()` comparison.



Comparison

▨ `MyFloor()`

- ✓ User-defined function.
- ✓ Does not use the built-in `floor()` function.
- ✓ Handles positive and negative numbers differently.
- ✓ Uses integer conversion to remove the fractional part.

▤ `floor()`

- ✓ Built-in C++ function.
- ✓ Directly calculates the floor value.
- ✓ Correctly handles both positive and negative numbers.
- ✓ Used in this program only to compare the result with `MyFloor()`.

✦ Summary

- ✓ Read a floating-point number.
- ✓ If positive, convert it directly to `int`.
- ✓ If negative, decrease it by `1` before converting it to `int`.
- ✓ Return the floor value.
- ✓ Positive `33.54323` becomes `33`.
- ✓ Negative `-33.54323` becomes `-34`.
- ✓ Compare the custom result with C++'s `floor()` function.

Thank you 😊